

CSE344 – Systems Programming

Homework #2

procSearch Report

Name: Ayşe Feyza SERBEST
Student ID: 220104004052
Course: CSE344
Homework: #2
Date: March 27, 2026

Contents

1	Introduction	2
2	Design Decisions	2
2.1	Directory Partitioning	2
2.2	Signal Handling Design	2
2.3	Avoiding Async-Signal-Safety Issues	2
2.4	Regex Implementation	3
3	Screenshots	3
3.1	Test Case 1	3
3.2	Test Case 2	3
3.3	Test Case 3	4
3.4	Test Case 4	4
3.5	Test Case 5	4
3.6	Test Case 6	5
4	Conclusion	5

1. Introduction

This program is a multi-process file search utility written in C. It searches files inside a directory using multiple worker processes and reports matching files based on a given pattern and optional size filter.

The main goal of the design is to efficiently utilize multiple processes while maintaining correct inter-process communication and safe signal handling.

2. Design Decisions

2.1. Directory Partitioning

Directories are divided among worker processes using a round-robin approach. This ensures that the workload is distributed evenly regardless of directory order.

2.2. Signal Handling Design

Signal handling was one of the most challenging parts of this project.

Handling signals asynchronously requires careful design because most standard functions are not safe to use inside signal handlers.

To solve this problem:

- Global variables were used to share state between the main program and signal handlers.
- Only async-signal-safe functions such as `write()` were used.
- Workers respond to `SIGTERM` by sending partial results before exiting.

Using global variables significantly simplified communication between the signal handler and the main execution flow.

2.3. Avoiding Async-Signal-Safety Issues

To avoid undefined behavior:

- No `printf()` was used inside signal handlers.
- Only `write()` system call was used.
- Shared variables were defined as `volatile sig_atomic_t`.

2.4. Regex Implementation

The regex functionality was another challenging part.

Instead of using a full regex library, a simplified custom matching logic was implemented.

The main difficulty was handling repetition patterns like `c+`, which required careful recursive matching.

3. Screenshots

3.1. Test Case 1

```
keyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$ ./procSearch -d /tmp/crawltest -n 3 -f 'repo+rt'
[Worker PID:9804] MATCH: /tmp/crawltest/beta/report_final.txt (14 bytes)
[Worker PID:9802] MATCH: /tmp/crawltest/alpha/sub1/repoort.txt (21 bytes)
[Worker PID:9802] MATCH: /tmp/crawltest/alpha/report.txt (22 bytes)
[Worker PID:9803] MATCH: /tmp/crawltest/gamma/repooot.txt (16 bytes)
/tmp/crawltest
----- alpha
----- sub1
----- repoort.txt (21 bytes) [Worker 9802]
----- report.txt (22 bytes) [Worker 9802]
----- gamma
----- repooot.txt (16 bytes) [Worker 9803]
----- beta
----- report_final.txt (14 bytes) [Worker 9804]
--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 4
Worker PID 9802 : 2 matches
Worker PID 9803 : 1 match
Worker PID 9804 : 1 match
```

Figure 1: Regex matching

3.2. Test Case 2

```
keyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$ ./procSearch -d /tmp/crawltest -n 3 -f 'repo+rt' -s 15
[Worker PID:9808] MATCH: /tmp/crawltest/alpha/sub1/repoort.txt (21 bytes)
[Worker PID:9808] MATCH: /tmp/crawltest/alpha/report.txt (22 bytes)
[Worker PID:9809] MATCH: /tmp/crawltest/gamma/repooot.txt (16 bytes)
/tmp/crawltest
----- alpha
----- sub1
----- repoort.txt (21 bytes) [Worker 9808]
----- report.txt (22 bytes) [Worker 9808]
----- gamma
----- repooot.txt (16 bytes) [Worker 9809]
--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 3
Worker PID 9808 : 2 matches
Worker PID 9809 : 1 match
Worker PID 9810 : 0 matches
```

Figure 2: 15 bytes

3.3. Test Case 3

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$ ./procSearch -d /tmp/crawltest -n 3 -f 'repo+rt' -s 20
[Worker PID:9818] MATCH: /tmp/crawltest/alpha/sub1/repoort.txt (21 bytes)
[Worker PID:9818] MATCH: /tmp/crawltest/alpha/report.txt (22 bytes)
/tmp/crawltest
|----- alpha
|----- sub1
|----- repoort.txt (21 bytes) [Worker 9818]
|----- report.txt (22 bytes) [Worker 9818]
--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 2
Worker PID 9818 : 2 matches
Worker PID 9819 : 0 matches
Worker PID 9820 : 0 matches
```

Figure 3: 20 bytes

3.4. Test Case 4

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$ ./procSearch -d /tmp/crawltest -n 3 -f 'xyz+123'
No matching files found.
--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 0
Worker PID 9857 : 0 matches
Worker PID 9858 : 0 matches
Worker PID 9859 : 0 matches
```

Figure 4: No matching files found.

3.5. Test Case 5

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$ ./procSearch -d "/usr" -n 3 -f xyz+123
No matching files found.
--- Summary ---
Total workers used : 3
Total files scanned : 103045
Total matches found : 0
Worker PID 10682 : 0 matches
Worker PID 10683 : 0 matches
Worker PID 10684 : 0 matches
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$ ps -ef | grep procSearch
feyza      10686    9491  0 19:58 pts/1    00:00:00 grep procSearch
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$
```

Figure 5: Avoid zombie processes.

3.6. Test Case 6

```
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$ ./procSearch -d /usr -n 3 -f 'readme' 2>> monitor.log &
PID=$!
sleep 1
kill -INT $PID
sleep 1
cat monitor.log
[1] 10890
[Worker PID:10893] MATCH: /usr/share/mime/text/x-readme.xml (3185 bytes)
[Worker PID:10893] MATCH: /usr/share/alsa/ucm/README.md (198 bytes)
[Worker PID:10893] MATCH: /usr/share/alsa/ucm2/README.md (1461 bytes)
[Worker PID:10893] MATCH: /usr/share/dict/README.select-wordlist (199 bytes)
[Worker PID:10893] MATCH: /usr/share/perl/5.32.1/Encode/README.e2x (553 bytes)
[Worker PID:10893] MATCH: /usr/share/snmp/mib2c-data/mfd-readme.m2c (30434 bytes)
[Worker PID:10893] MATCH: /usr/share/snmp/mib2c-data/syntax-DateAndTime-readme.m2i (288 bytes)
[Worker PID:10893] MATCH: /usr/share/vim/vim82/doc/README.Debian (324 bytes)
[Worker PID:10892] MATCH: /usr/lib/ispell/README.select-ispell (197 bytes)
[Worker PID:10893] MATCH: /usr/share/themes/Default-xhdpixfwm4/README (179 bytes)
[Worker PID:10893] MATCH: /usr/share/themes/Moheli/xfwm4/README (252 bytes)
[Worker PID:10893] MATCH: /usr/share/themes/Default-hdpxfwm4/README (179 bytes)
[Worker PID:10892] MATCH: /usr/lib/x86_64-linux-gnu/perl5/5.32/TK/README.Adjust (3359 bytes)
[Worker PID:10892] MATCH: /usr/lib/x86_64-linux-gnu/perl/debian-config-data-5.32.1/README (2039 bytes)
[Worker PID:10893] SIGTERM received. Partial matches: 10 (5996 files). Exiting.
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$ ^C
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$
--- Summary ---
Total workers used : 3
Total files scanned : 7691
Total matches found : 10
Worker PID 10892 : 0 matches
Worker PID 10893 : 10 matches
Worker PID 10894 : 0 matches
^C
[1]+  Exit 1                  ./procSearch -d /usr -n 3 -f 'readme' 2>> monitor.log
feyza@debian11:~/Desktop/GTU-CSE344-Systems-Programming/HW2$ cat monitor.log
[Parent] SIGINT received. Terminating all workers...
```

Figure 6: monitor.log

4. Conclusion

This project helped deepen my understanding of:

- Process management with `fork()`
- Inter-process communication using pipes
- Signal handling in Unix systems

The most difficult parts were signal handling and regex implementation.

Signal handling was particularly challenging due to async-signal-safety constraints. Using global variables helped manage shared state safely.

Regex implementation required careful thinking to correctly match patterns with repetition.

Despite these challenges, the project provided valuable experience in system-level programming.